

CSI 2110
Data Structures and Algorithms

Lecture 2: Stacks, Queues and Deques

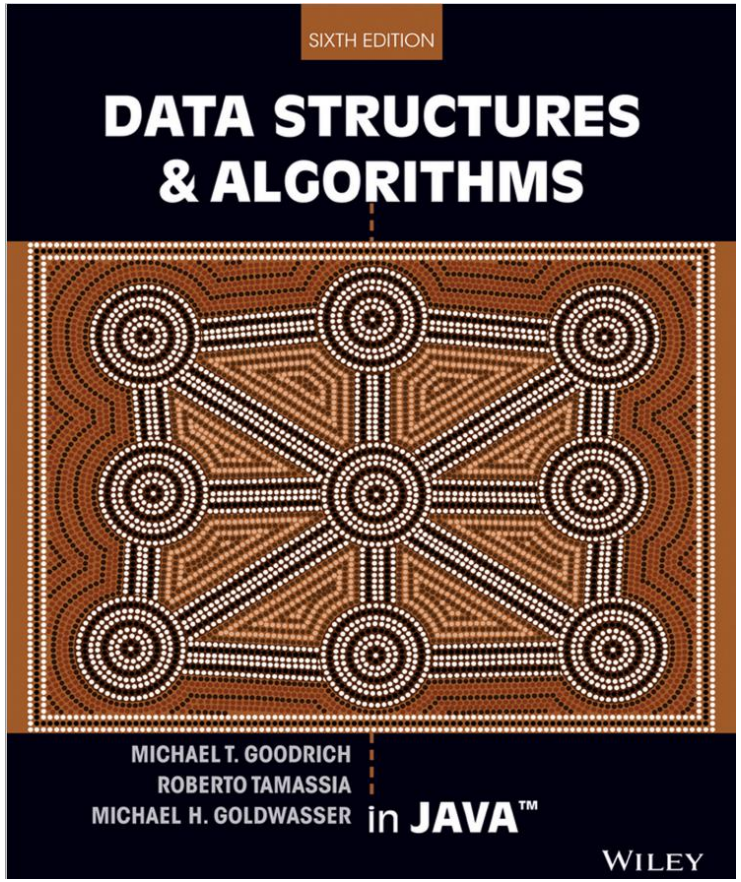
Bo Sun

Fall 2026

A note on the slides

Adapted from the slides that accompany this book.

Part of the slides is based on previous slides from Prof. Lucia Moura for CSI 2110.



Data Structures and Algorithms in Java
6th edition
Michael Goodrich, Roberto Tamassia, Michael Goldwasser
Wiley, 2014

Stacks, Queues, and Deques: roadmap

- Abstract data type (ADT)
- Stack
- Queue
- Deque

Abstract Data Types (ADT)

- Abstract data type (ADT) is an abstract view of a data structure. The ADT specifies
  - what can be stored in the ADT
  - what operations can be supported by the ADT
- The ADT specifies what each operation does, but NOT how it does it
  - In Java an ADT can be expressed by an interface

```
public interface myList {

   public void addLast(int e);

   public void print();
}
```

Concrete Data Structure

- An ADT is implemented/realized by a concrete data structure
  - In Java, a concrete data structure implements an ADT in the same way as a class implements an interface

```
public class SLList implements myList {
    /** A nested node class */
    public Node head;

    public SLList(int e) {
        head = new Node(e, null);
    }

    public void addLast(int e) {
        Node current = head;
        while(current.next != null) {
            current = current.next;
        }
        current.next = new Node(e, null);
    }

    public void print() {
        Node current  = head;
        while(current != null) {
            System.out.print(current.element +
" ");
            current = current.next;
        }
        System.out.println();
    }
}
```

```
public class DLList implements myList {
    /** a nested node class */
    public Node head;
    public Node tail;

    public DLList(int e) {
        head = tail = new Node(null, e, null);
    }

    public void addLast(int e) {
        tail.next = new Node(tail, e, null);
        tail = tail.next;
    }

    public void print() {
        Node p = head;
        while(p != null) {
            System.out.print(p.element + " ");
            p = p.next;
        }
        System.out.println();
    }
}
```

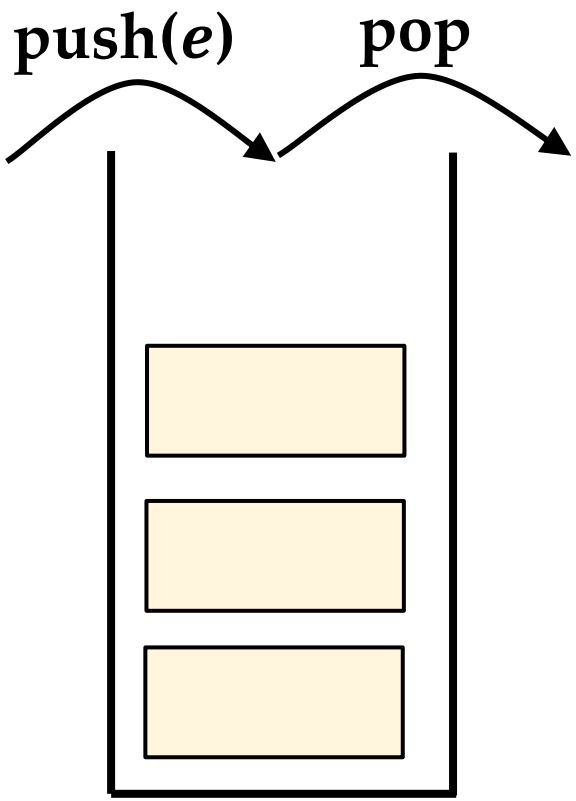
Stacks, Queues, and Deques: roadmap

- Abstract data type (ADT)
- Stack
- Queue
- Deque

Stack

- A stack is a collection of elements that are inserted and deleted according to last-in first-out (LIFO) principle





Main methods

- push(e): add element *e* to the top of the stack
- pop(): remove and return the top element from the stack

Stack ADT

Main methods

- push(e): add element *e* to the top of the stack
- pop(): remove and return the top element from the stack (or null if the stack is empty)

Support methods

- size(): return the number of elements of the stack
- isEmpty(): return a Boolean indicating whether the stack is empty
- top(): return the top element of the stack without removing it (or null if the stack is empty)

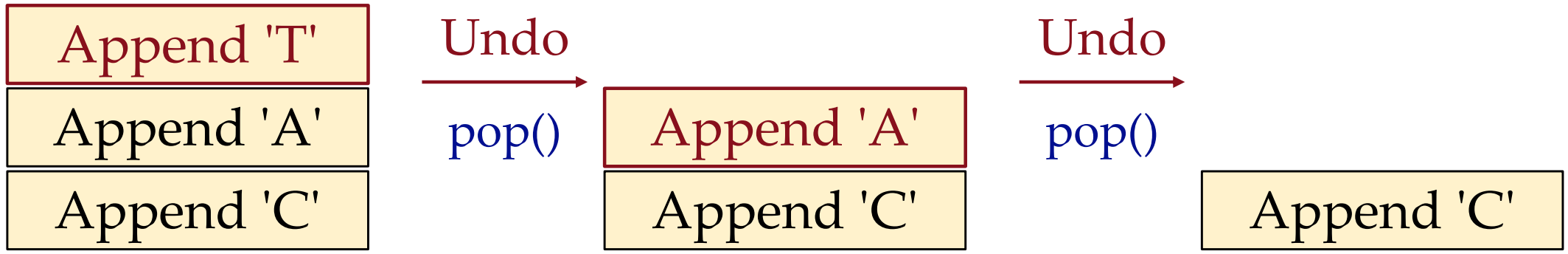
Applications of stacks: Undo operations

- Suppose each editing action appends one character
  - Type C, A, T sequentially
  - Then what happens when you press Undo twice

After typing
Text: **CAT**

Undo once
Text: **CA**

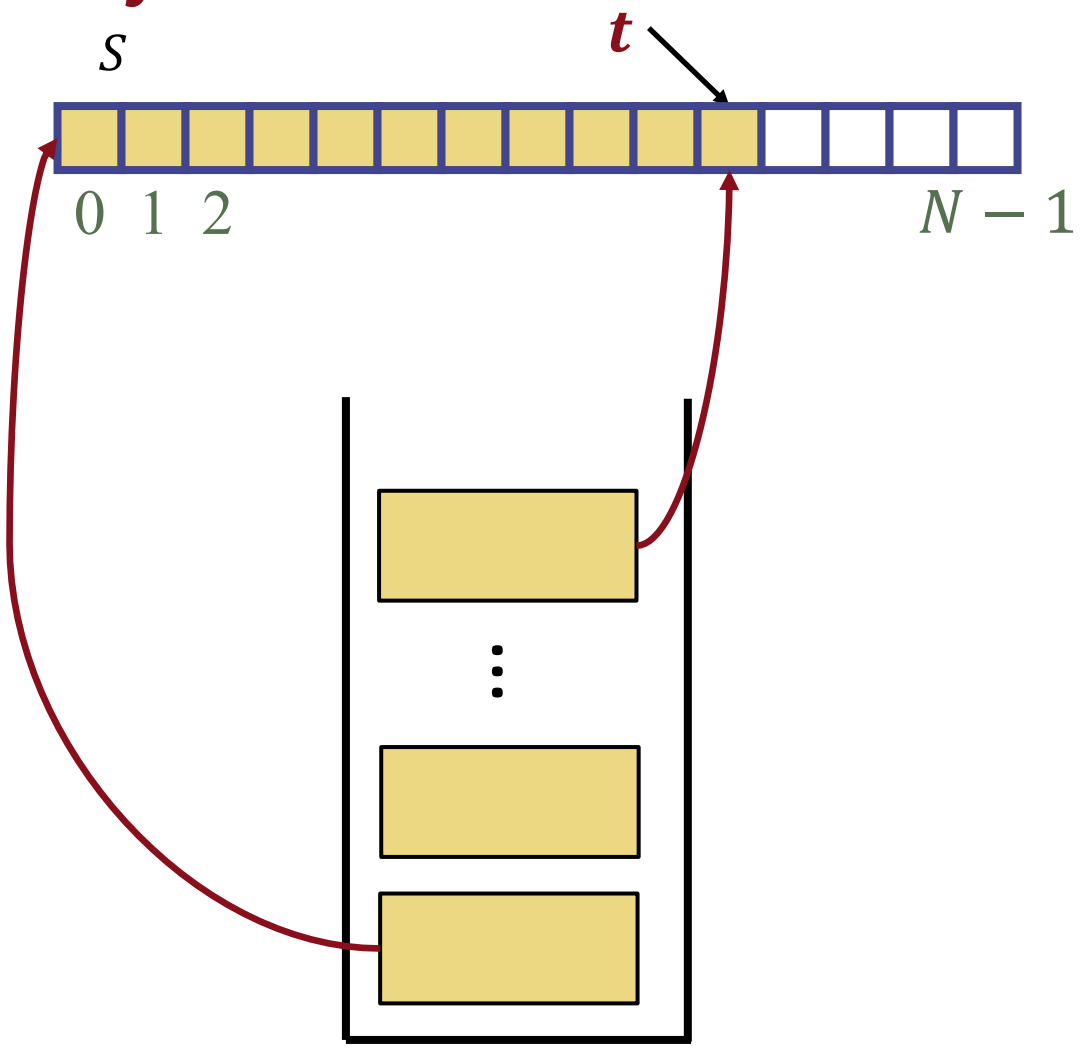
Undo again
Text: **C**



The most recent action is undone first: **Last-In, First-Out (LIFO)**

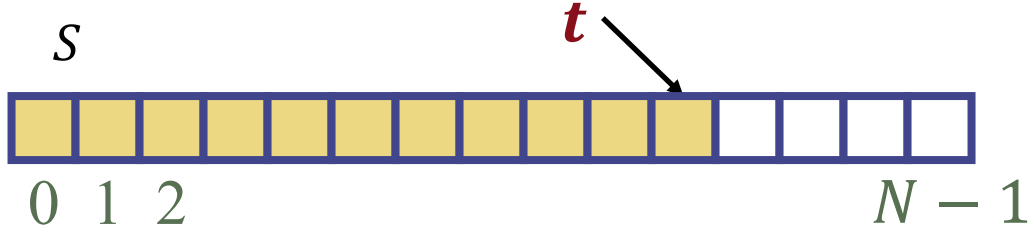
Implementing a stack with an array

- The stack consists of an N-element array S and an integer variable t (the index of the top element in array)



Implementing a stack with an array

- The stack consists of an N-element array S and an integer variable t (the index of the top element in array)




```
S ← new array of N elements
t ← −1    ▷ empty stack
Algorithm size()
    return t + 1
Algorithm isEmpty()
    return (t = −1)
Algorithm top()
    If (isEmpty()) then
      return null
    Else
      return S[t]
```

```
Algorithm push(e)
    If (size()=N) then
       Error
    Else
       t ← t + 1
       S[t] ← e
Algorithm pop()
    If (isEmpty()) then
      return null
    Else
      temp ← S[t]
      t ← t − 1
      return temp
```

Array-based stack: Performance

- Time and space efficiency

Time:

size()	$O(1)$
isEmpty()	$O(1)$
top()	$O(1)$
push(e)	$O(1)$
pop()	$O(1)$

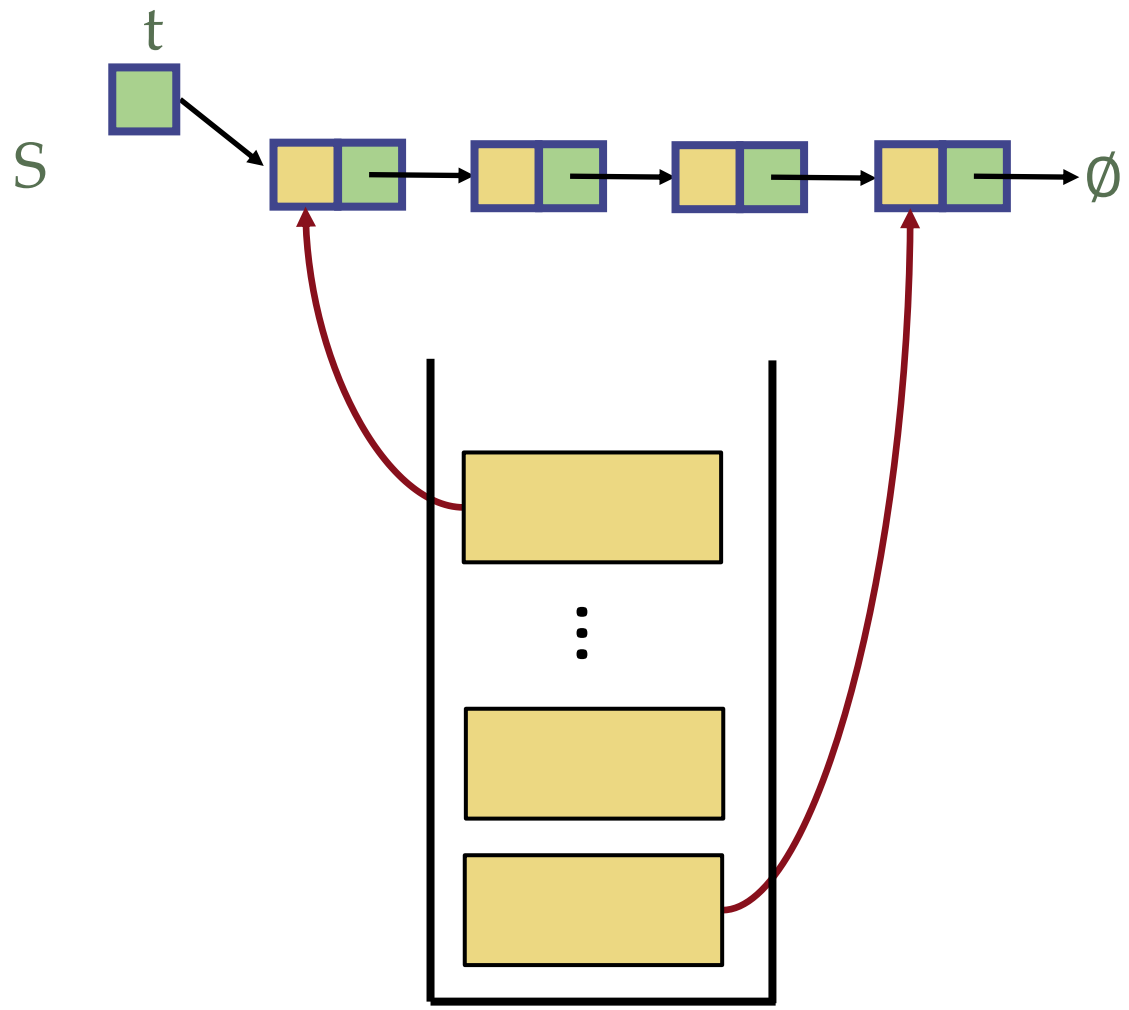
Space: $O(N)$

N = size of the array

- Limitation: static structure
  - solution: extendible array (to be covered in the list ADT)

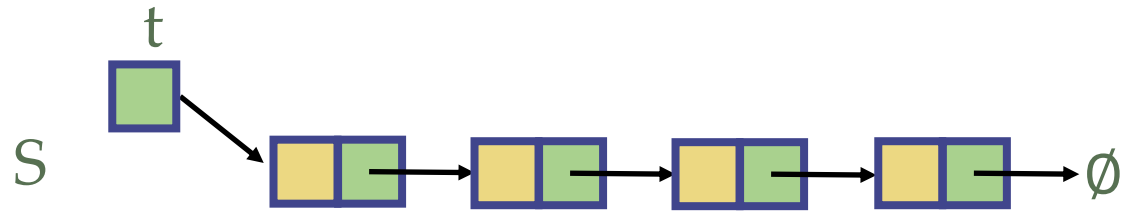
Implementing a stack with a Singly Linked List

- The stack consists of a singly linked list *S* and an integer variable *sz* (the current size of the list)



Implementing a stack with a Singly Linked List

- The stack consists of a singly linked list S and an integer variable sz (the current size of the list)




```
t ← head pointer of linked list
sz ← 0                    ▷ empty stack

Algorithm size()
    return sz

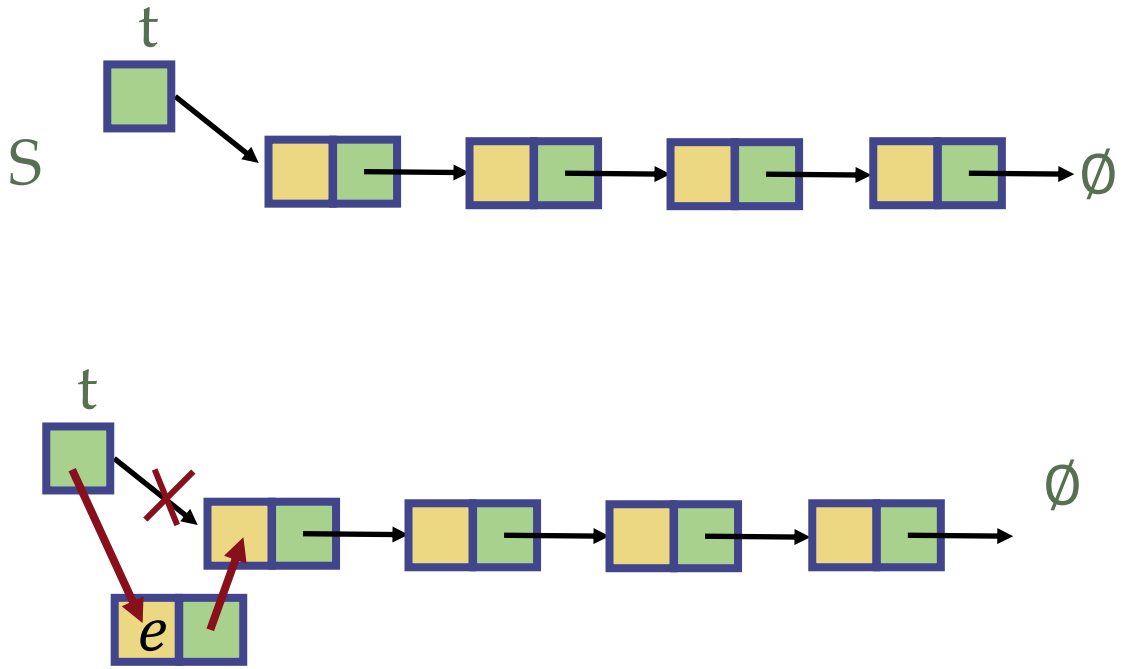
Algorithm isEmpty()
    return (sz = 0)

Algorithm top()
    If (isEmpty()) then
      return null
    Else
      return t.element
```

Implementing a stack with a Singly Linked List

- The stack consists of a singly linked list S and an integer variable sz (the current size of the list)

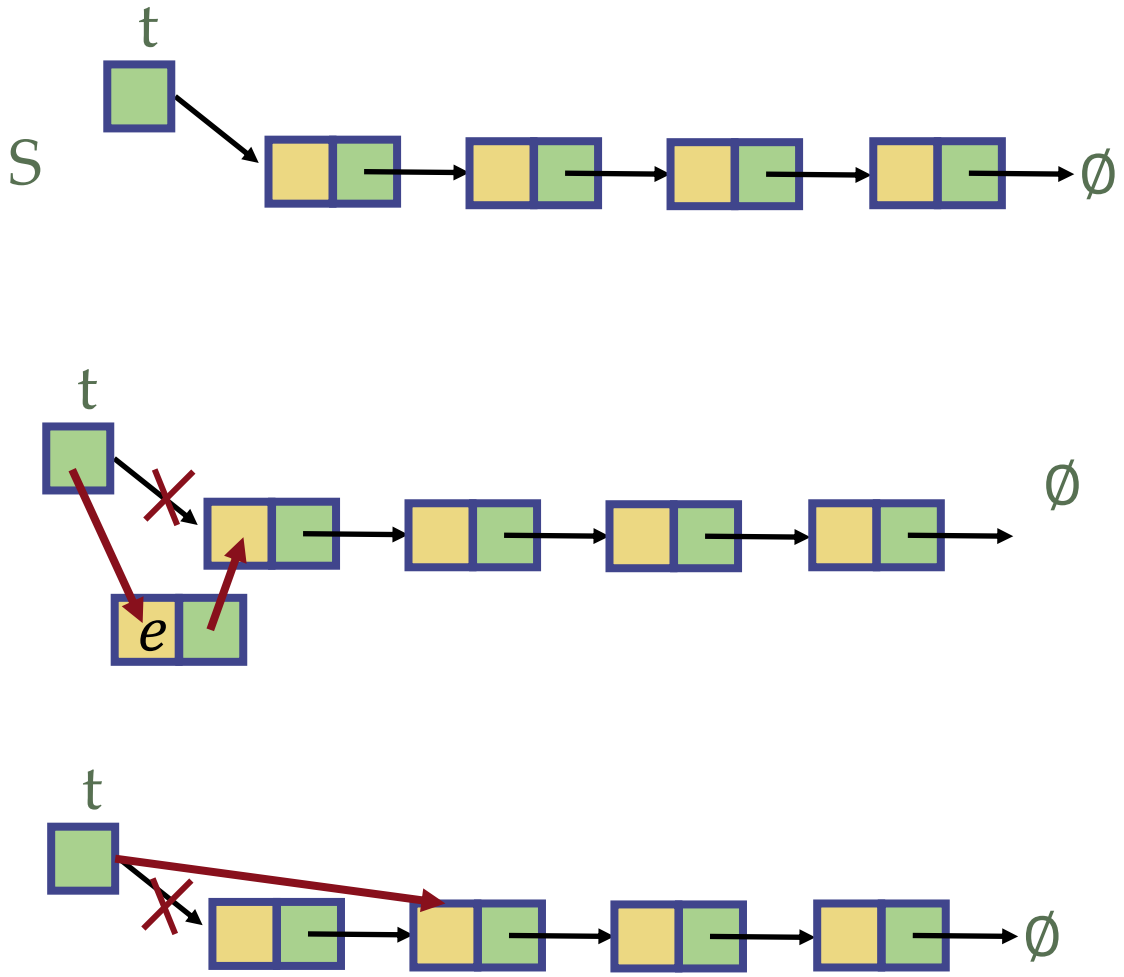
```
Algorithm push(e)
    n ← new Node
    n.element ← e
    n.next ← t
    t ← n
    sz ← sz + 1
```



Implementing a stack with a Singly Linked List

- The stack consists of a singly linked list S and an integer variable sz (the current size of the list)

```
Algorithm pop()
    If (isEmpty()) then
      return null
    Else
      temp ← t.element
      t ← t.next
      sz ← sz − 1
      return temp
```



SLList-based stack: Performance

- Time and space efficiency

Time:

size()	$O(1)$
isEmpty()	$O(1)$
top()	$O(1)$
push(e)	$O(1)$
pop()	$O(1)$

Space: $O(n)$

n = number of the elements

ADT for stack: Our own stack interface in Java

- Java interface corresponding to our Stack ADT
  - Different from the built-in Java class java.util.Stack
- Assumes null is returned from pop() and top() when stack is empty

```
public interface Stack<E> {
   int size();
   boolean isEmpty();
   E top();
   void push(E element);
   E pop();
}
```

Concrete data structure: our own array-based stack in Java

concrete data structure

```
public class ArrayStack<E>
      implements Stack<E> {

  // holds the stack elements
  private E[ ] S;

  // index to top element
  private int top = -1;

  // constructor
  public ArrayStack(int capacity) {
      S = (E[ ]) new Object[capacity];
   }
```

Stack ADT

```
   public E pop() {
     if (isEmpty())
            return null;
      E temp = S[top];
      // facilitate garbage collection:
      S[top] = null;
      top = top – 1;
      return temp;
    }
   //other methods of stack interface
   public int size() { return (top + 1);}
   public boolean isEmpty() { return
(top == -1);}
   public void push (E element) {  /*
code here*/ }
   public E top() {/* code here*/ }
}
```

Example 1: reversing an array

- Input: an array *X* of elements
- Output: the elements in *X* are reversed

Input:
[4, 1, 2, 3]

Output:
[3, 2, 1, 4]

```
public class ReverseAlgorithms {

 public static void intReverse (Integer[] a, int num) {
  Stack<Integer> s;
  s = new ArrayStack<Integer>(num);
  for (int i=0; i<num; i++) {s.push(a[i]);}
  for (int i=0; i<num; i++) {a[i]=s.pop();}
 }
```

```
  public static void main(String[] args) {

    int[] list[]={4,1,2,3};
    intReverse(list,list.length);

    //… code that prints list  …

  }

}
```

Example 2: Parenthesis Matching

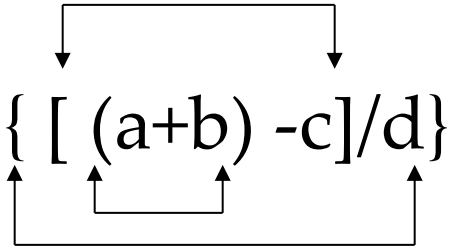
- Input: a string *X* with parenthesis
- Output: true if all parenthesis are matching and false otherwise

Each "(", "{", or "[" must be paired with a matching ")", "}", or "]"

- ([ ] { }) ← True
- ([ )] ← False
- (( ) ← False
-) (← False

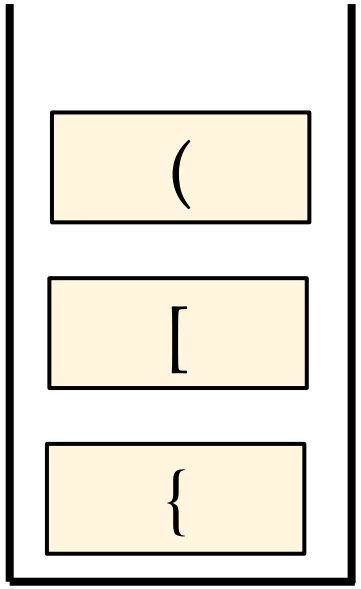
Parenthesis matching using a stack

Checking for balanced parenthesis



True

{ [a+b) -c]/d} False



{ [(a+b) -c]/d}

when we read an opening parenthesis, push it into stack
when we read a closing parenthesis input=')'
pop() ='(' from stack and check they have matching types

Parenthesis Matching in Java

```
public static boolean isMatched(String expression) {

  final String opening = "({["; // opening delimiters
  final String closing = ")}]"; // respective closing delimiters
  Stack<Character> buffer = new LinkedStack<Character>( );

  for (char c : expression.toCharArray( )) {
    if (opening.indexOf(c) != −1) // this is a left delimiter
      buffer.push(c);
    else if (closing.indexOf(c) != −1) { // this is a right delimiter
      if (buffer.isEmpty( )) // nothing to match with
        return false;
      if (closing.indexOf(c) != opening.indexOf(buffer.pop( )))
        return false; // mismatched delimiter
      }
    }
  return buffer.isEmpty( ); // were all opening delimiters matched?
}
```

Example 3: Evaluating arithmetic expressions

- Input: an arithmetic expression
- Output: the value of the expression

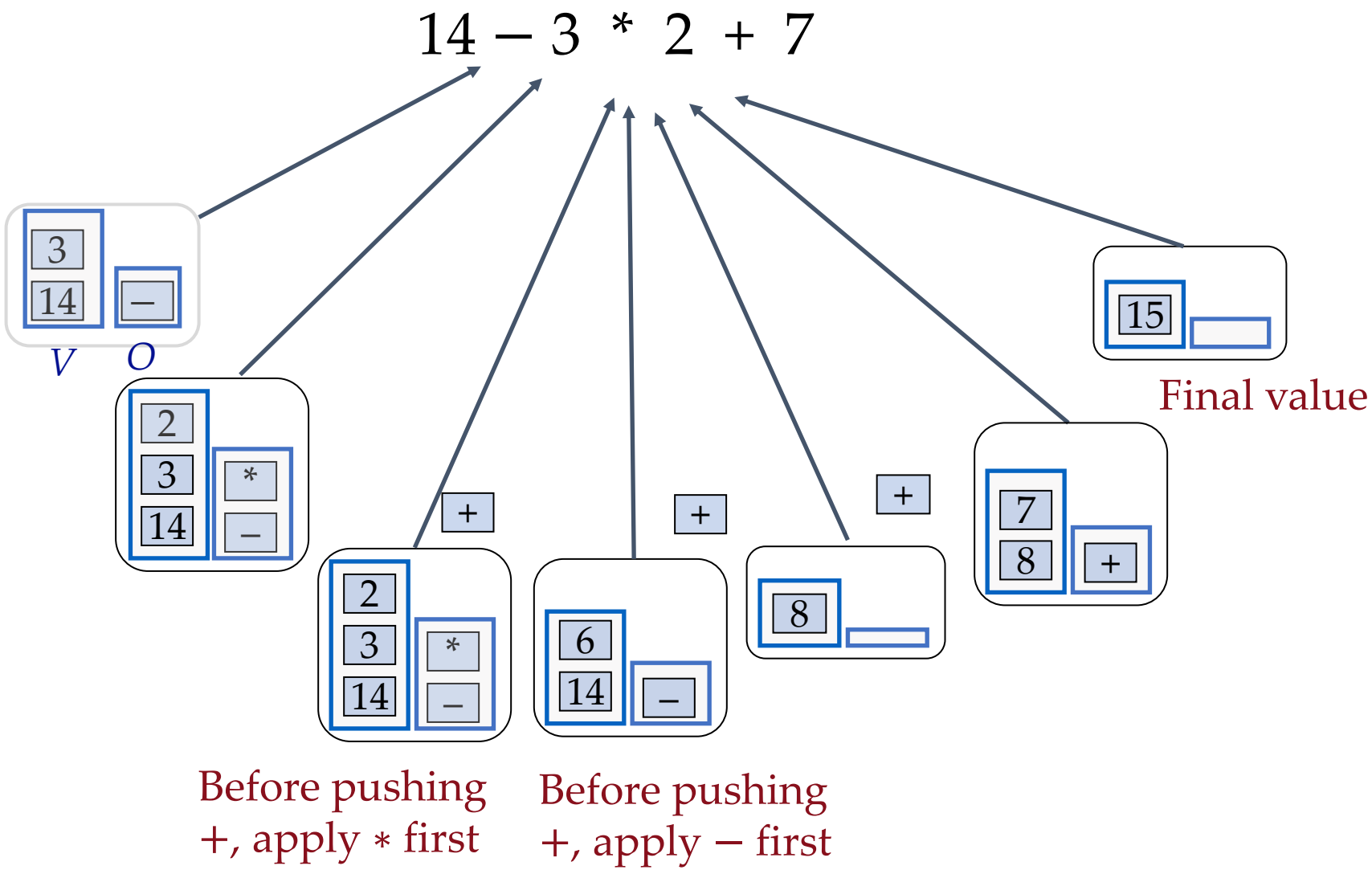
$$14 - 3 * 2 + 7 = (14 - (3 * 2)\ ) + 7$$

Evaluation order
* has precedence over +/–
Equal precedence: left to right

Two stacks
V: values and partial results
O: operators waiting to be applied

Idea: **before** pushing an operator onto stack *O*, evaluate the operators with *higher* and *equal* precedence

Evaluating arithmetic expressions using stack



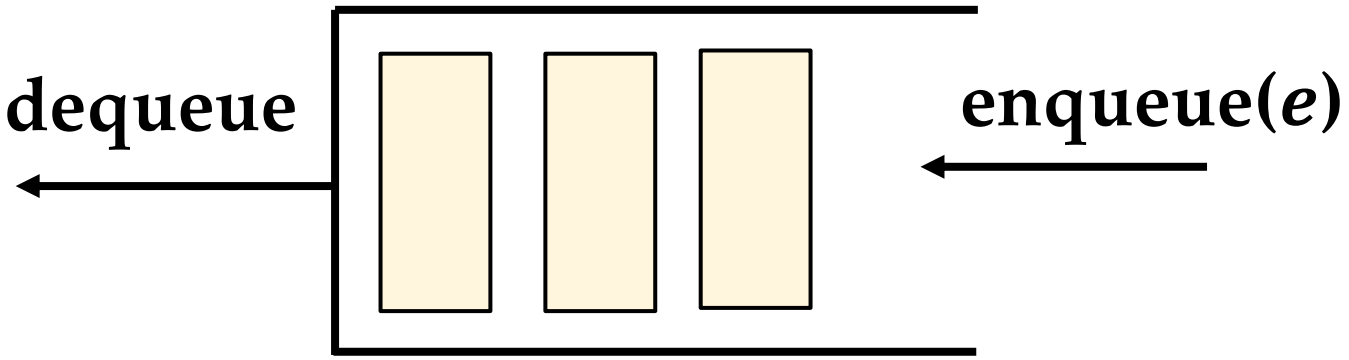
Stacks, Queues, and Deques: roadmap

- Abstract data type (ADT)
- Stack
- Queue
- Deque

Queue

- A queue is a collection of elements that are inserted and deleted according to first-in first-out (FIFO) principle





Main methods

- enqueue(e): add element *e* to the rear of the queue
- dequeue(): remove and return the element from front of the queue

Queue ADT

Main methods

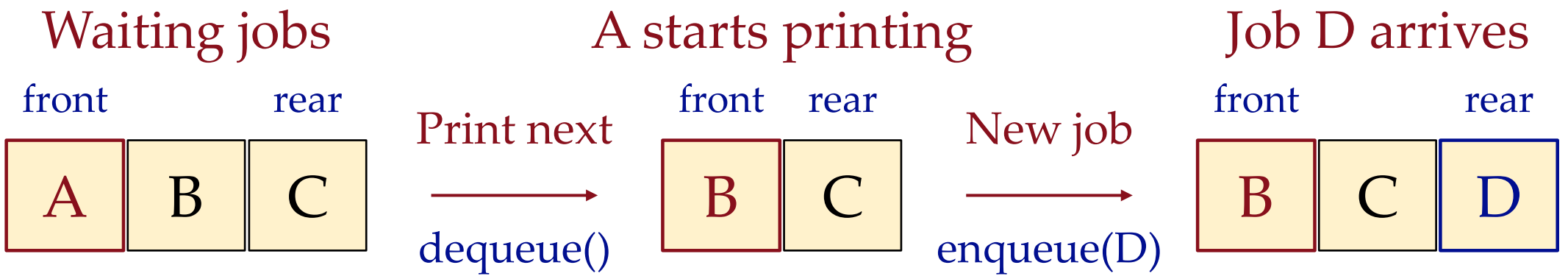
- enqueue(e): add element *e* to the rear of the queue
- dequeue(): remove and return the element from the front of the queue (or null if the queue is empty)

Support methods

- size(): return the number of elements in the queue
- isEmpty(): return a Boolean indicating whether the queue is empty
- first(): return the front element without removing it (or null if the queue is empty)

Applications of Queues: A shared printer

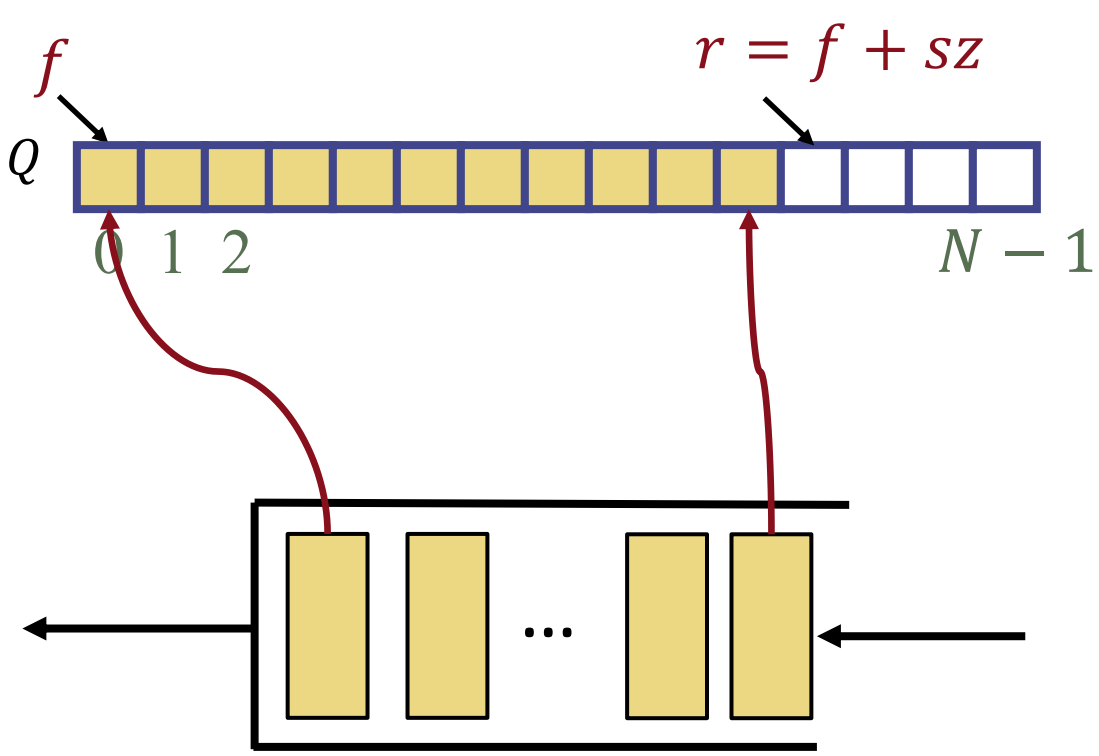
- Suppose a shared printer processes waiting jobs in arrival order.
  - Jobs A, B, and C arrive while it is busy.
  - Then which job should print next?

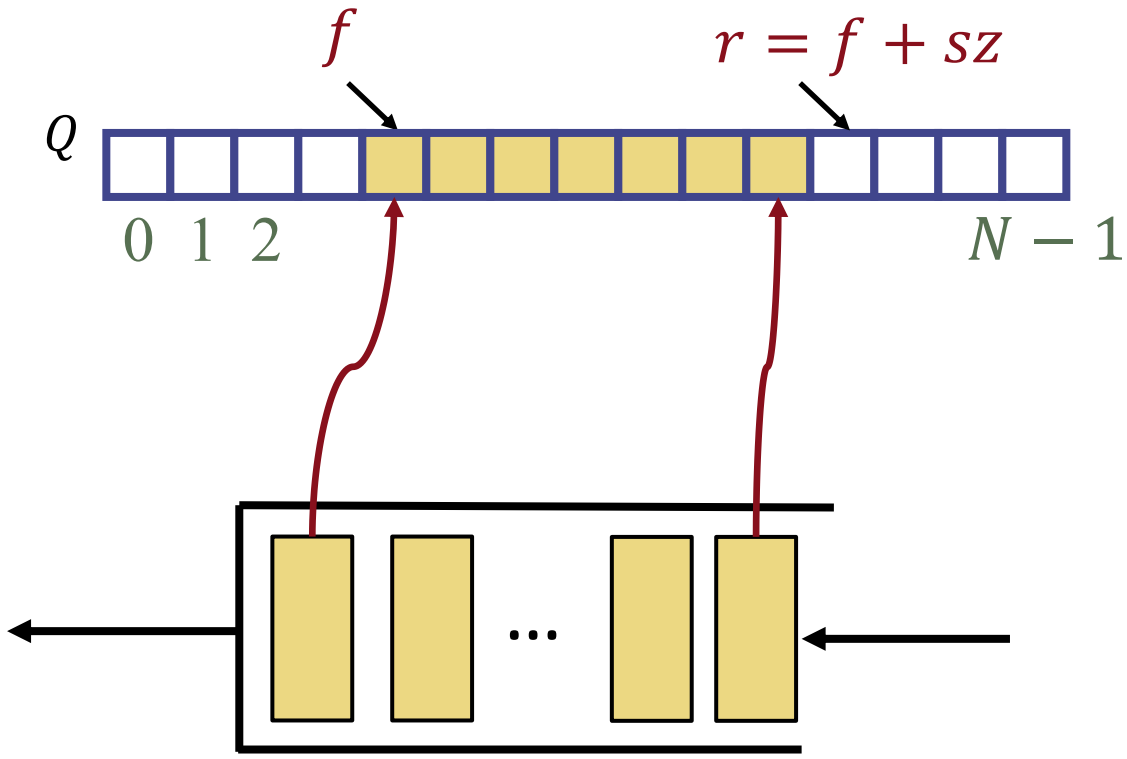


Serve the earliest waiting job first: **First-In, First-Out (FIFO)**.

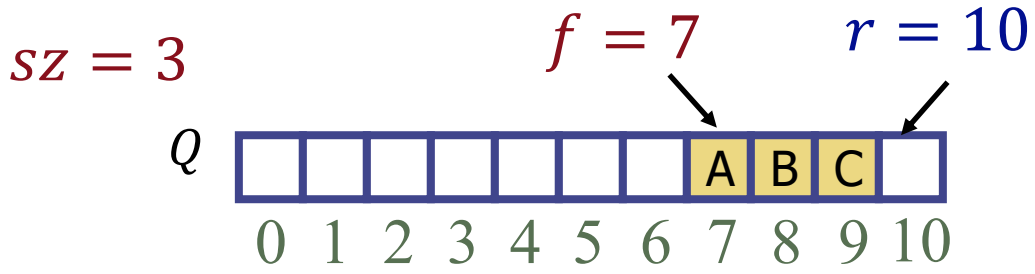
Implementing a queue with an array

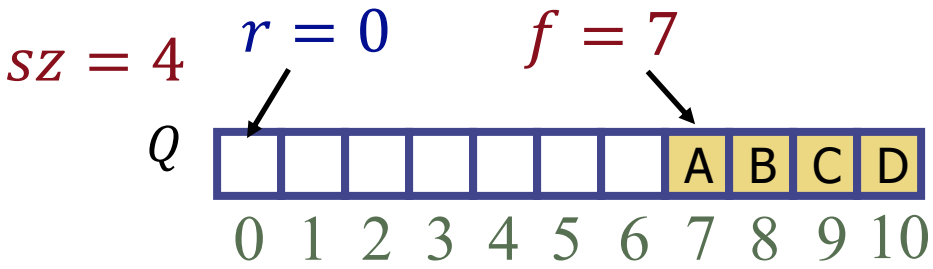
- The queue consists of an N-element array Q and two integer variables
  - f: index of the front element
  - sz: number of stored elements



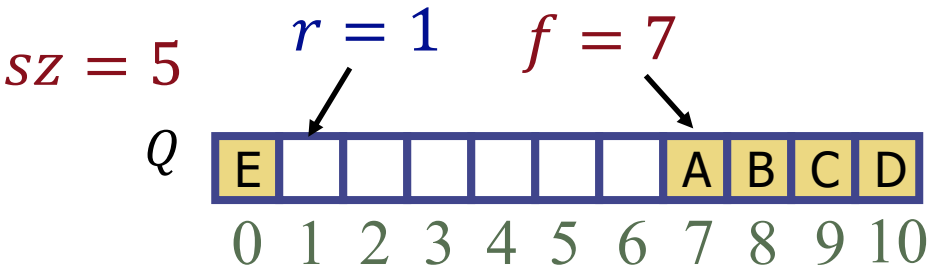


Circular array

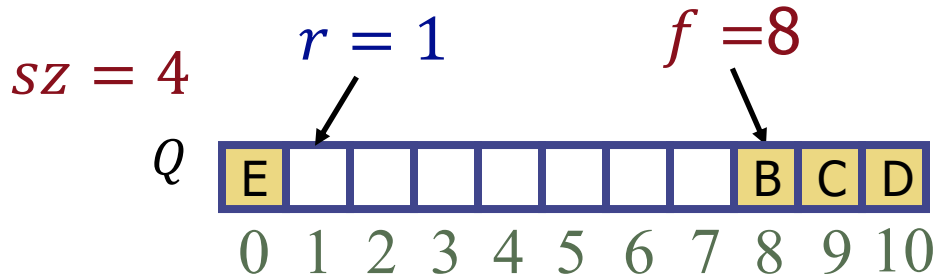




enqueue(D)



enqueue(E)



dequeue()

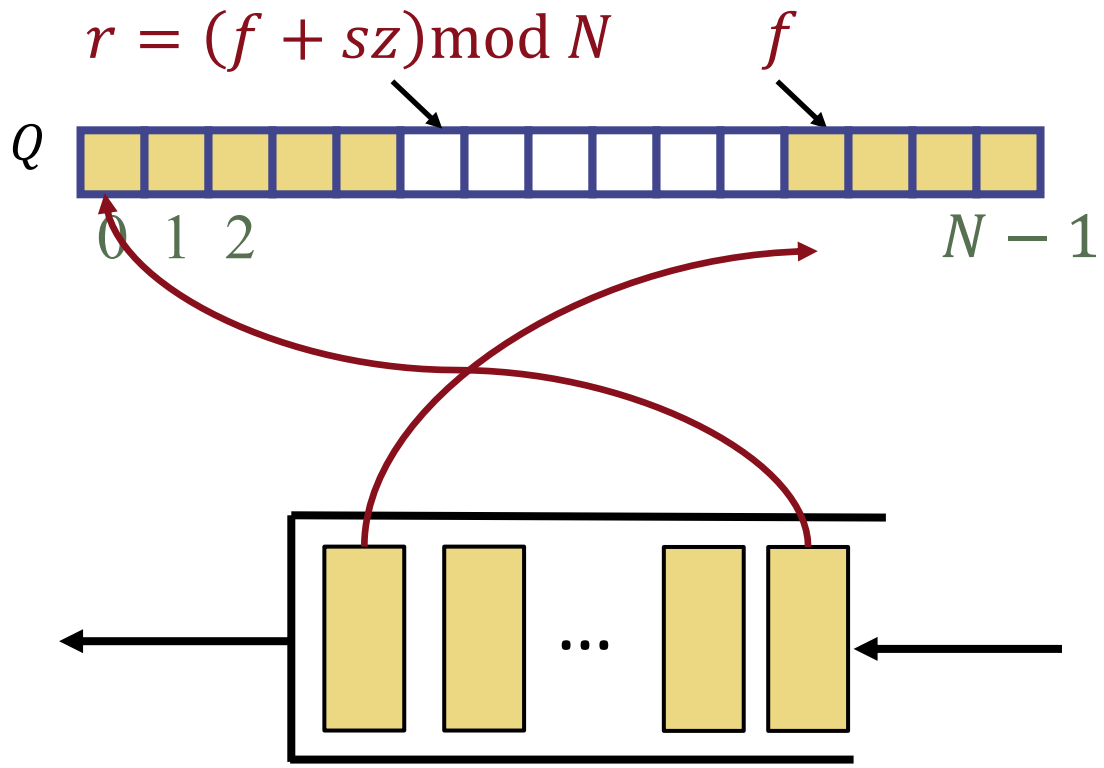
- Array capacity $N = 11$
- Perform operations
  - enqueue(D)
  - enqueue(E)
  - dequeue()

Next insertion index

$$r = (f + sz) \bmod N$$

Implementing a queue with an array

- The array is in a circular fashion
- $r = (f + sz) \bmod N$ is the index after the rear of the queue
- $(f + sz - 1) \bmod N$ is the index of the rear



- When is the queue empty?
  - ~~$f = r$~~ or $sz = 0$
- When is the queue full?
  - ~~$f = r$~~ or $sz = N$

Implementing a queue with an array

```
Q ← new array of N elements
f ← 0; sz ← 0
Algorithm size()
    return sz
Algorithm isEmpty()
    return (sz = 0)
Algorithm first()
    If (isEmpty()) then
      return null
    Else
      return Q[f]
```

```
Algorithm enqueue(e)
    If (sz = N) then
       Error
    Else
       r ← (f + sz) mod N
       Q[r] ← e
       sz ← sz + 1
Algorithm dequeue()
    If (isEmpty()) then
      return null
    Else
      temp ← Q[f]
      f ← (f + 1) mod N
      sz ← sz − 1
      return temp
```

Array-based queue: Performance

- Time and space efficiency

Time:

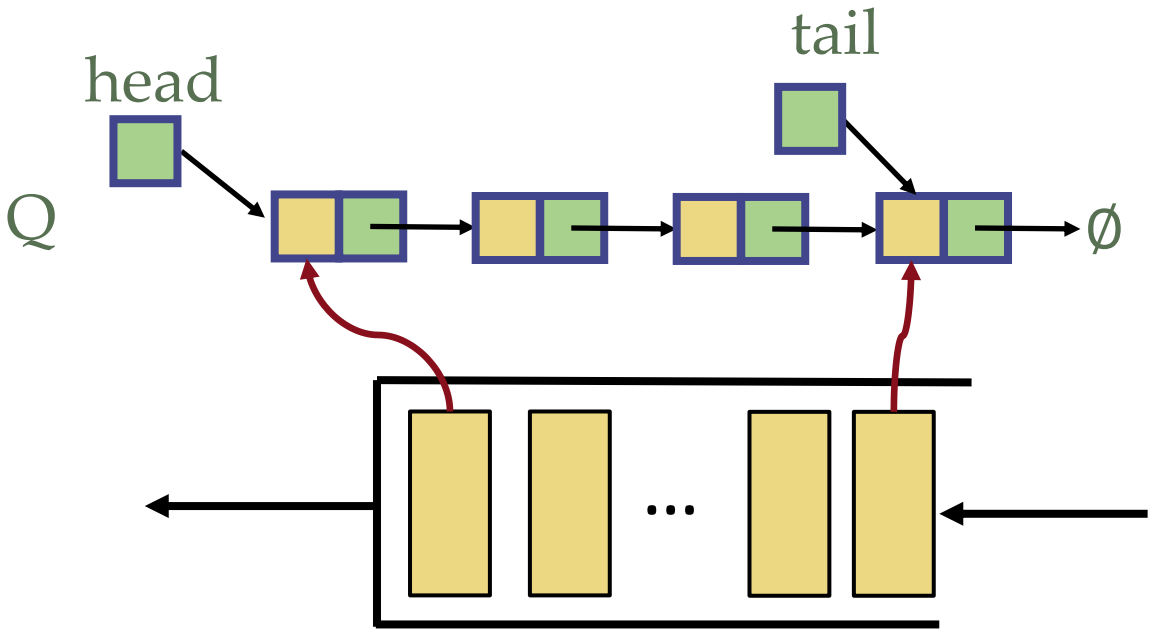
size()	$O(1)$
isEmpty()	$O(1)$
first()	$O(1)$
enqueue(e)	$O(1)$
dequeue()	$O(1)$

Space: $O(N)$

N = size of the array

Implementing a queue with a Singly Linked List

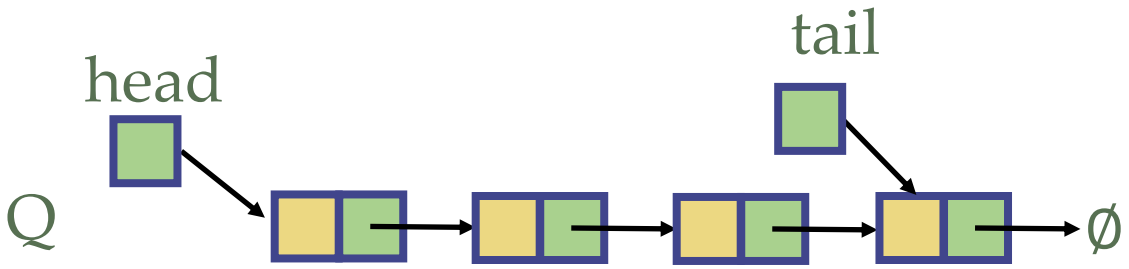
- The queue consists of a singly linked list Q with one pointer to the head and one pointer to the tail
  - The head of the list is the front of the queue
  - The tail of the list is the rear of the queue



Implementing a queue with a Singly Linked List

- The queue consists of a singly linked list Q with one pointer to the head and one pointer to the tail
  - The head of the list is the front of the queue
  - The tail of the list is the rear of the queue

Why not do the opposite?



- Inserting and deleting at the head are both $O(1)$
- Inserting at the tail is $O(1)$
- Deleting at the tail is $O(n)$!!

List-based queue: Performance

- Time and space efficiency

Time:

size()	$O(1)$
isEmpty()	$O(1)$
first()	$O(1)$
enqueue(e)	$O(1)$
dequeue()	$O(1)$

Space: $O(n)$

n = number of stored elements

Our Queue Interface in Java

- Java interface corresponding to our Queue ADT
- Assumes that first() and dequeue() return null if queue is empty

```
public interface Queue<E> {
  int size();
  boolean isEmpty();
  E first();
  void enqueue(E e);
  E dequeue();
}
```

Array-based queue vs. List-based queue

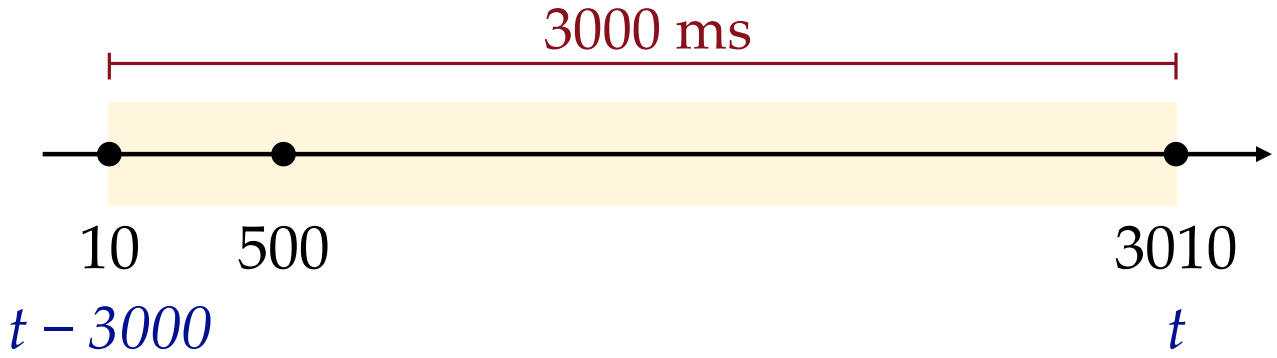
- If we know in advance a reasonable upper bound for the number of elements in the queue, then use array-based implementation

- Otherwise
  - List-based implementation
  - Extendable array-based implementation

Example: Number of Recent Calls

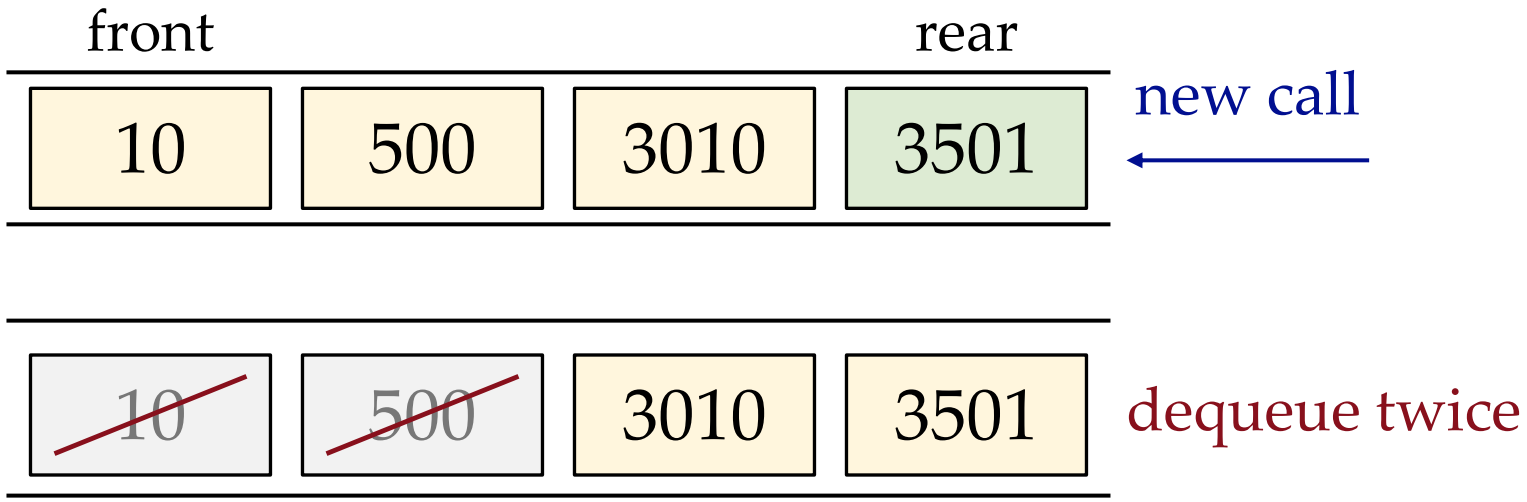
- Input: calls arrive at increasing times t (milliseconds)
- Output: ping(t) returns the number of calls in $[t - 3000, t]$

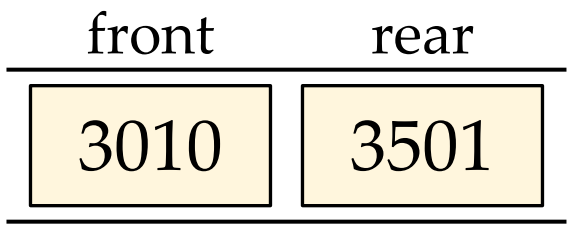
Call	Output
ping(10)	1
ping(500)	2
ping(3010)	3
ping(3501)	2



Idea: store timestamps in a queue, oldest at the front.

Counting recent calls using a queue





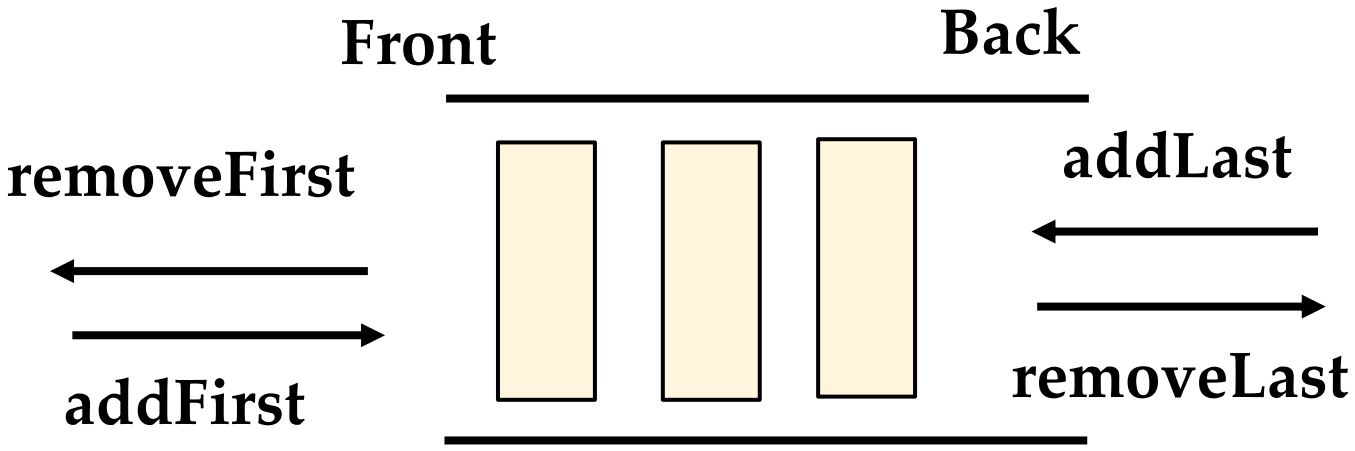

```
Algorithm ping(t)
  Q.enqueue(t)
  cutoff ← t − 3000
  while not Q.isEmpty() and Q.first() < cutoff do
    Q.dequeue()
  return Q.size()
```

Stacks, Queues, and Deques: roadmap

- Abstract data type (ADT)
- Stack
- Queue
- Deque

Deque

- A deque is a Double-Ended Queue, that can insert and remove elements from both the front and back



Deque is more general than both stack and queue

Main methods

- addFirst(e): add element *e* to the front of the deque
- removeFirst(): remove and return the element from front of the deque
- addLast(e): add element *e* to the back of the deque
- removeLast(): remove and return the element from back of the deque

Deque ADT

Main methods

- addFirst(e): add element *e* to the front of the deque
- removeFirst(): remove and return the element from front of the deque (or null if the deque is empty)
- addLast(e): add element *e* to the back of the deque
- removeLast(): remove and return the element from back of the deque (or null if the deque is empty)

Support methods

- size(): return the number of elements in the deque
- isEmpty(): return a Boolean indicating whether the deque is empty
- first(): return the first element without removing it (or null if the deque is empty)
- last(): return the last element without removing it (or null if the deque is empty)

Implementing Stacks and Queues with Deques

Stacks with Deques:

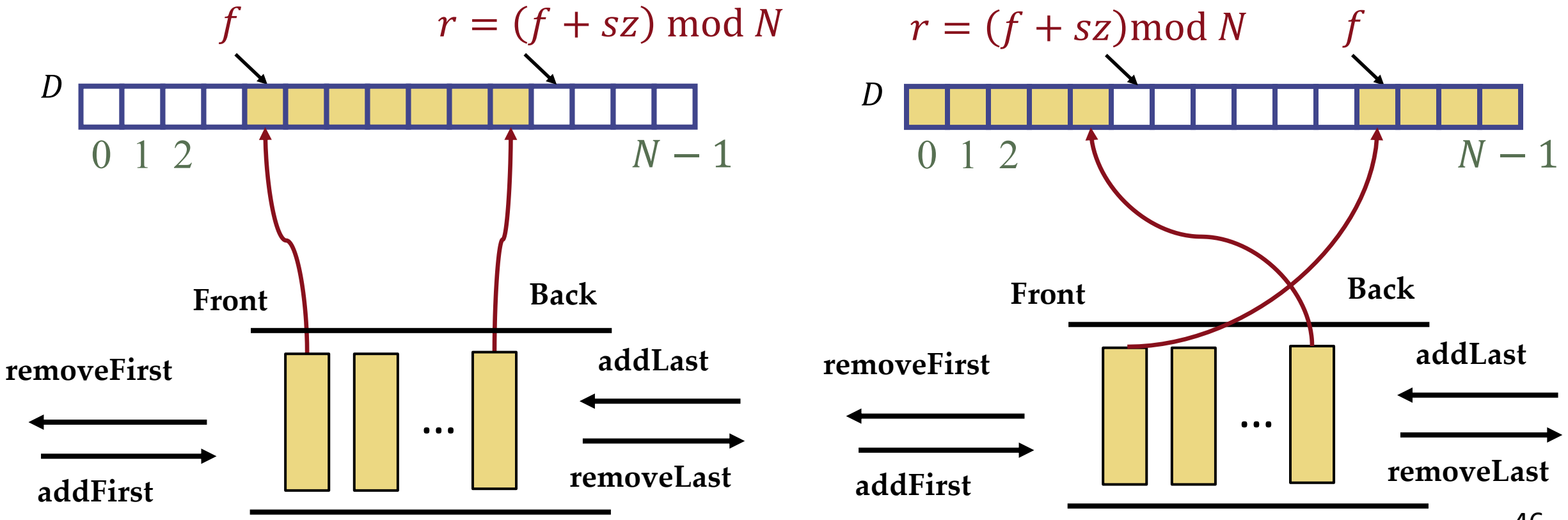
Stack Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
top()	last()
push(e)	addLast(e)
pop()	removeLast()

Queues with Deques:

Queue Method	Deque Implementation
size()	size()
isEmpty()	isEmpty()
first()	first()
enqueue(e)	addLast(e)
dequeue()	removeFirst()

Implementing a deque with a circular array

- The deque consists of an N-element array D and two integer variables
  - f: index of the front element
  - sz: number of stored elements



Implementing a deque with a circular array

$D \leftarrow$ new array of N elements

$f \leftarrow 0$; $sz \leftarrow 0$

Algorithm addFirst(e)

```
If (sz = N) then
    Error
Else
    f ← (f − 1 + N) mod N
    D[f] ← e
    sz ← sz + 1
```

Algorithm addLast(e)

```
If (sz = N) then
    Error
Else
    r ← (f + sz) mod N
    D[r] ← e
    sz ← sz + 1
```

Algorithm removeFirst()

```
If (isEmpty()) then
    return null
Else
    temp ← D[f]
    f ← (f + 1) mod N
    sz ← sz − 1
    return temp
```

Algorithm removeLast()

```
If (isEmpty()) then
    return null
Else
    last ← (f + sz − 1) mod N
    temp ← D[last]
    sz ← sz − 1
    return temp
```

Array-based deque: Performance

- Time and space efficiency

Time:

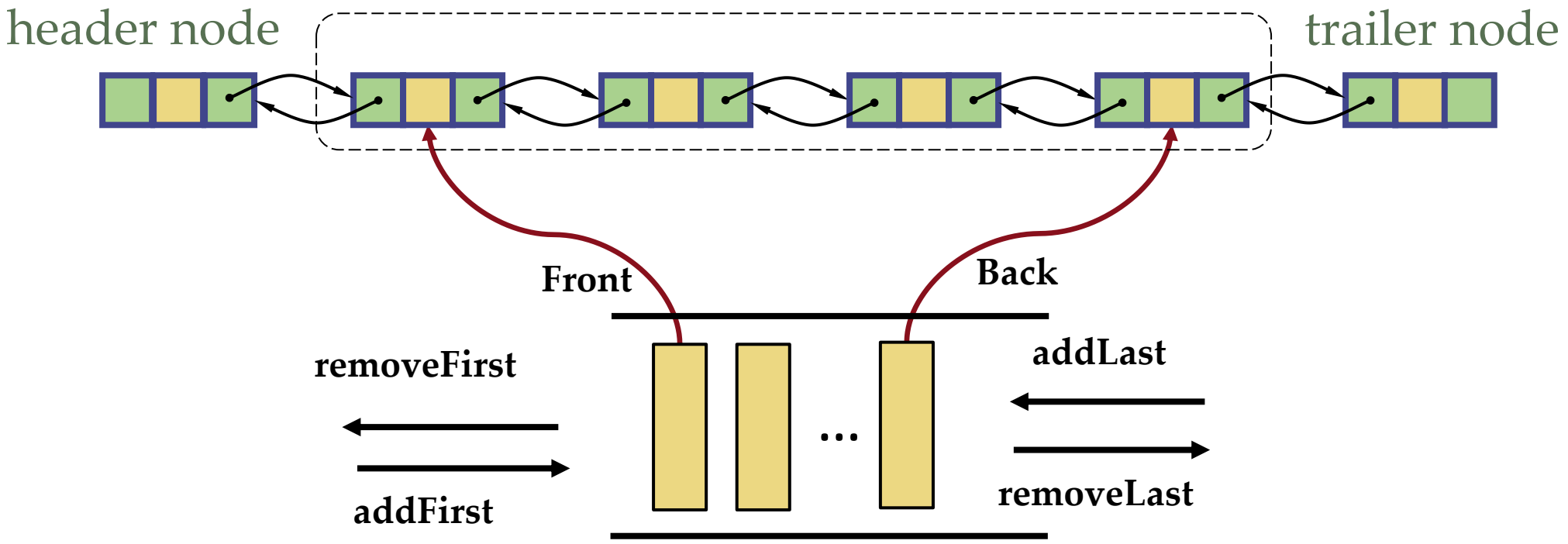
size()	$O(1)$
isEmpty()	$O(1)$
first()	$O(1)$
last()	$O(1)$
addFirst(e)	$O(1)$
removeFirst()	$O(1)$
addLast(e)	$O(1)$
removeLast()	$O(1)$

Space: $O(N)$

N = size of the array

Implementing a deque with a doubly linked list

- The deque consists of a doubly linked list with two dummy nodes
  - The header node is before the first element. It has a valid next pointer but a null prev pointer
  - The trailer node is after the last element. It has a valid prev pointer and a null next pointer



List-based deque: Performance

- Time and space efficiency

Time:

size()	$O(1)$
isEmpty()	$O(1)$
first()	$O(1)$
last()	$O(1)$
addFirst(e)	$O(1)$
removeFirst()	$O(1)$
addLast(e)	$O(1)$
removeLast()	$O(1)$

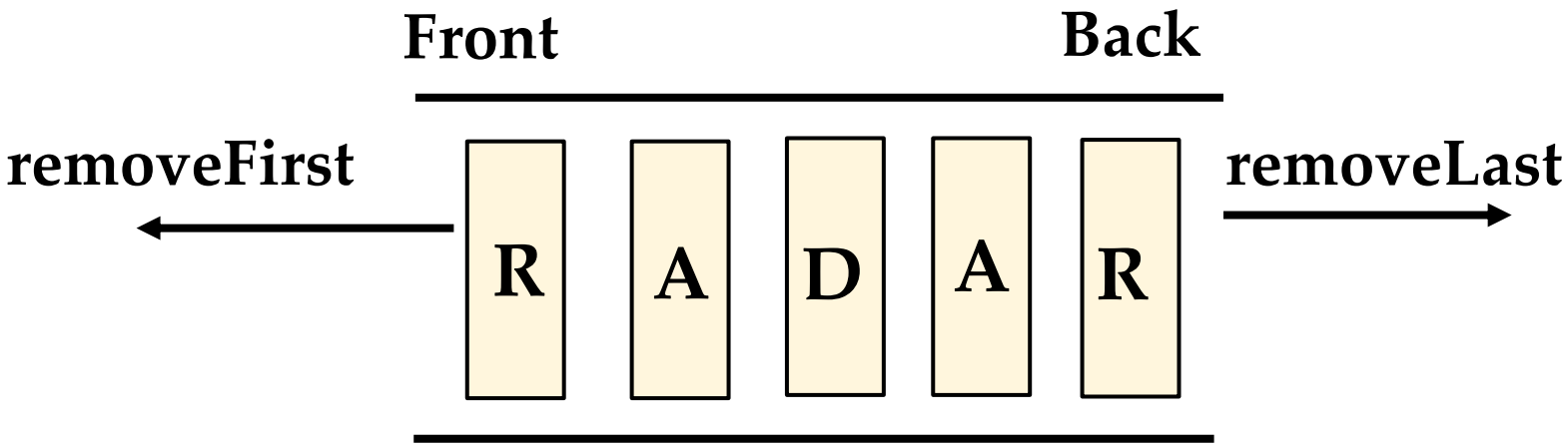
Space: $O(n)$

n = number of the elements

Example: Palindromes using deque

- Input: a string *X*
- Output: true if the string is a palindrome, and false otherwise

- Idea: read the string into a deque, and then compare the outputs from its front and back



Exercise: Stack and queue operations

- Consider the following piece of code that uses an empty stack (mystatck) and an empty queue (myqueue). Show the elements in the stack at points //1, //2, //3

```
for (i = 0; i <= 4; i++){
    mystack.push(i);
    myqueue.enqueue(i);
}
//1
for (i=0; i <= 3; i++)
     myqueue.enqueue(mystack.pop());
//2
mystack.push(myqueue.dequeue());
mystack.push(myqueue.dequeue());
//3
```

Next Topic

lists